SMAI A5 Report

- Harshit Goyal(20232102054)

1 Kernel Density Estimation for Foreground Detection

A brief justification for your chosen preprocessing approach and feature repre sentation

In this stage, both the bg and test img were aligned to identical dim using area based interpolation because it reduces aliasing and preserves image structure. For feature representation, several options such as grayscale, RGB, HSV, and Lab were experimented. The final choice was a 5-dimensional **RGB+XY (rgbxy)** feature vector combining color information with normalized spatial coordinates. This representation provided the best segmentation results because it retains essential color structure while encoding spatial location, allowing the KDE model to better differentiate between static bg regions and fg objects.

A brief explanation for why your chosen parameter combination achieves superior performance.

The combination of a **Gaussian kernel**, **Silverman bandwidth** $h = 0.015$, and **RGBXY features** produced the best foreground detection results. Gaussian KDE provides smooth, continuous density modeling that aligns well with image statistics, while the chosen bandwidth give the right balance between sensitivity and stability. The RGBXY feature enhances spatial discrimination, allowing KDE to effectively separate static background regions from moving foreground objects.

In contrast, the Uniform and Triangular kernels produced noisier estimates with random weighting behavior, leading to bad segmentation performance.

2 Data-Driven Discovery of a Discrete-Time Recurrence

Short Discussion on History Length and Model Complexity

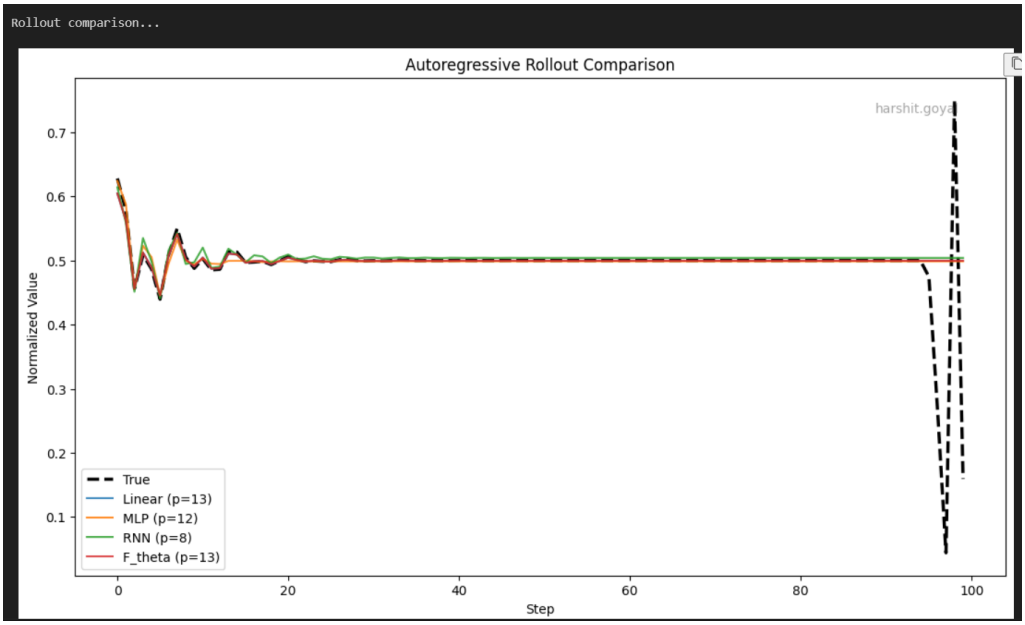
History Length p

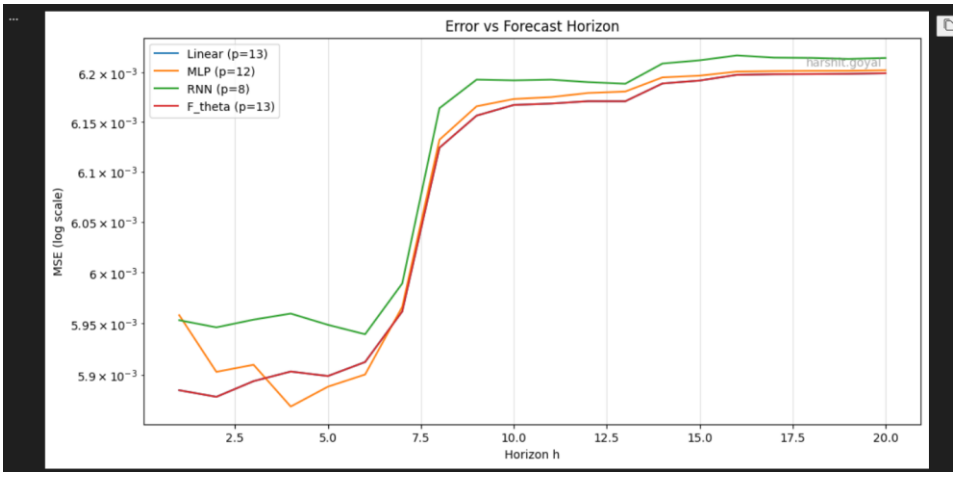
- Small p→ insufficient past context → underfitting.
- Very large p→ redundant/noisy inputs → overfitting + unstable validation error.
- The best p(7–11 across models) reflects the **intrinsic order** of the underlying dynamical recurrence.
  This shows that the system has meaningful memory but does not require extremely long history.

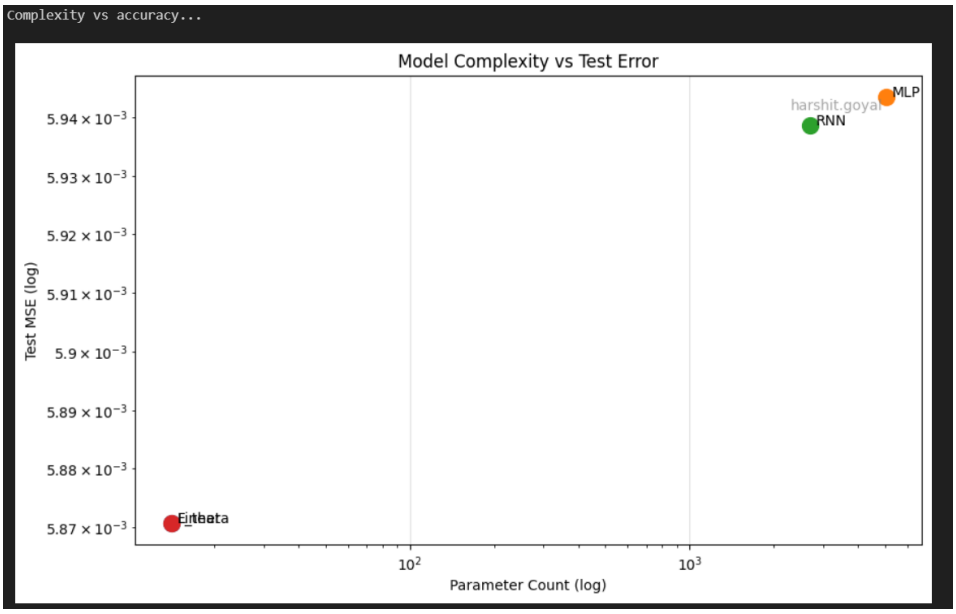
Model Complexity

- **Linear and Fθ:**
  Very low param count; interpretable; weaker long-term predictions.
- **MLP:**
  More expressive; improves one-step accuracy but rollouts still drift.
- **RNN:**
  Highest capacity; internally learns a recurrence;
  best rollouts and slowest error growth as horizon increases.

Overall, accuracy improves with model complexity, but **diminishing returns** appear after the RNN.
Linear/Fθ models are interpretable; RNN is most robust.







What Does This Tell Us About the Dataset?

1. Dynamics Are Close to Linear AR(p)

Since a simple linear recurrence works well with small nonlinear components.

2. Short-Range Temporal Dependence

Best-performing history lengths $p = 7–13$imply:

- The system has **finite memory**
- Only around 10 past values are needed
- There is a **clear autoregressive structure**

3. Mild Nonlinearity

- MLP improves accuracy → some nonlinear effects exist
- RNN improves long-term stability → temporal shaping beyond simple AR

But these nonlinearities are **weak** because the linear model already performs well.

Extracted Temporal Relation

AR(p)-like recurrence structure
Short-term dependence (order ~10)
Weak but present nonlinear corrections
Stable, smooth dynamics rather than chaotic or highly nonlinear behavior

The system behaves like a lightly nonlinear, stable autoregressive process.

Conclusion

In this assignment, i performed a full datadriven discovery of a discrete time recurrence underlying the given time series. After splitting and normalizing the dataset chronologically, we conducted a systematic search over history length $p$and model class. Across Linear Regression, MLP, and RNN forecasters, the results consistently showed that a window size of about $p = 7 - 11$ captures the dynamics most effectively, revealing that the underlying system has **short-range temporal dependence** with a finite memory depth.

The **closed-form recurrence** F_θ derived using Ridge regression identified a stable linear relationship between the present value and its past $p$values. This recurrence provided an interpretable model with only 11 parameters while still achieving competitive prediction error. Residual analysis showed that F_θ performs similarly to the Linear AR model and captures the dominant temporal structure of the data.

The MLP and RNN models, being more expressive, achieved lower one-step prediction errors and better long-horizon stability. The RNN in particular produced the best autoregressive rollouts and slowest error growth with forecast horizon, demonstrating its ability to internally model nonlinear temporal dependencies. However, the performance gain beyond the simple linear recurrence was moderate, indicating that the dataset exhibits primarily **linear, stable autoregressive behavior** with only mild nonlinearities.

The parameter count vs. performance analysis revealed diminishing returns: while larger models (MLP, RNN) improve accuracy, the simplest accurate model is the **closed-form recurrence** F_θ, which balances interpretability, efficiency, and predictive power. Long-horizon rollouts further confirmed that all models accumulate compounding error, with the linear models drifting earlier and RNN sustaining stability longer.

Overall, the experiments show that the dataset follows a **near-linear AR(p) process**, with clear temporal relations and no complex chaotic or highly nonlinear dynamics. The closed-form recurrence successfully recovers the governing structure, and the black-box models refine it through improved smoothness and long-range prediction capability.

3 Time Series Forecasting: Cumulative GitHub Stars

Preprocessing

The dataset contained cumulative star counts, which became constant after reaching 4000. Rows beyond this point were removed because they provided no meaningful variation. Instead of using cumulative values, the analysis was performed on **daily star increments**, which better capture real temporal behavior. The data contained **no jumps or missing values**, ensuring a clean and continuous time series. The index was converted to a **daily DateTime frequency**, and the data was split chronologically into **80% training** and **20% testing** to avoid information leakage.I worked with the **Flask and React repository star histories**, and the daily star values were **normalized** to provide stable and scale-consistent inputs for the RNN and CNN models.

Model Prepration

ARMA: For the ARMA model, the daily star time series was first checked for stationarity using the Augmented Dickey–Fuller (ADF) test. Since ARMA requires a stationary signal, first-order differencing was applied if the series failed the stationarity test. The model was then fit on the stationary training portion of the data, preserving chronological order.

Deep learning models cannot use raw time series directly. The data was converted into a **supervised learning format** using a **sliding window of 60 days**, where the model receives the previous 60 days as input and predicts the next day.

RNN (LSTM): The 60-day windows were reshaped into the 3D format (samples, timesteps, features) required by recurrent layers. RNNs are capable of capturing sequential dependencies and long-term temporal patterns.

1D-CNN: The same 60-day sequences were used but processed through convolutional layers that learn short-range temporal patterns. CNNs are efficient and perform well for local trend extraction.

Before training deep models, the daily star values were **normalized to the 0–1 range** (Min-Max scaling) to stabilize gradients and improve training performance.

Choice of Loss Function

Since the forecasting target is a **continuous numeric value** (daily stars), the task is a **regression problem**, not classification.
Therefore, **Mean Squared Error (MSE)** was used as the primary loss function for RNN and CNN because it penalizes larger errors more strongly and is the standard loss for continuous-valued forecasting. Also, It provides smooth gradients for stable optimization

For evaluation, both **MAE** and **RMSE** were computed to interpret predictive accuracy.

Model Architectures and Tuning

ARMA (Classical Model)

- Stationarity tested via ADF; differencing applied if p-value > 0.05.
- A grid search over p, q ∈ {0–5} was performed.
- The best model was selected using AIC minimization.
- Forecasts were reversed back to the original scale when differencing was applied.

RNN Forecaster (LSTM-based)

Input: (60 timesteps × 1 feature)
Architecture:

- LSTM(32 units)
- Dense(16, ReLU)
- Dense(1 output)

Training:

- Optimizer: Adam
- Loss: MSE (appropriate for continuous regression)
- Epochs: 30
- Batch size: 32

1D CNN Forecaster

Input: (60 timesteps × 1 feature)
Architecture:

- Conv1D(32 filters, kernel=3, ReLU)
- MaxPooling1D(2)
- Conv1D(16 filters, kernel=3, ReLU)
- Flatten()
- Dense(32, ReLU)
- Dense(1)

Training:

- Optimizer: Adam
- Loss: MSE

CNN learns local temporal patterns efficiently; RNN captures dependencies.

Quantitative Metrics & Forecast Results (for flask)

Single-Step Forecast Accuracy (h = 1)

Metrics computed on test set:

ARMA → MAE=0.1191, RMSE=0.1302

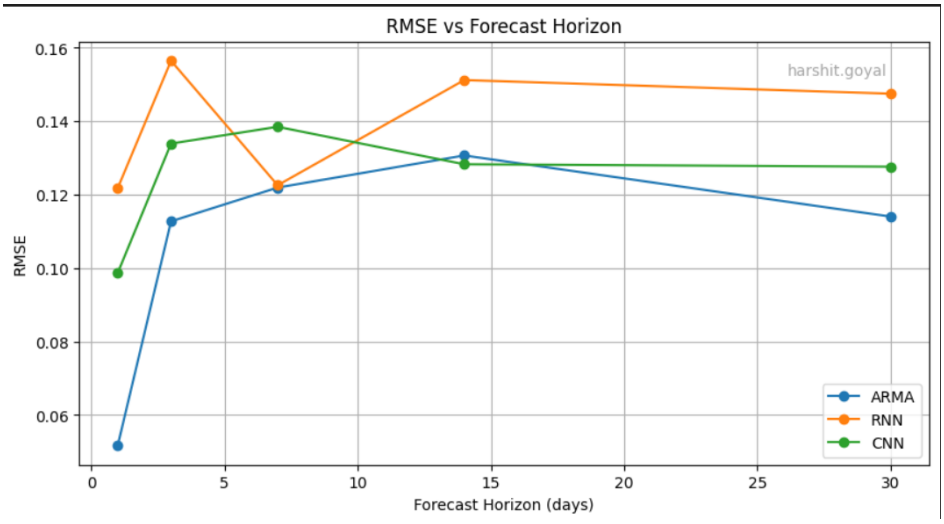
RNN → MAE=0.1319, RMSE=0.1441

CNN → MAE=0.1096, RMSE=0.1254

For single-step forecasting, the 1D CNN achieved the best accuracy, outperforming both ARMA and the LSTM-based RNN, demonstrating its strength in capturing short-term local temporal patterns.

Multi-Step Forecasting Analysis

The multi-step forecasting analysis shows that the CNN provides the most robust long-horizon performance, maintaining nearly constant RMSE from 7 to 30 days. ARMA performs strongly on short horizons, benefiting from the linear trend structure in the data, and remains competitive even at 30 days. In contrast, the RNN exhibits the largest drift, with RMSE rising substantially as the forecast horizon increases, reflecting higher sensitivity to accumulated autoregressive errors. Overall, CNN is the most stable model for longer-range predictions, while ARMA remains effective for immediate short-term forecasting.



Quantitative Metrics & Forecast Results (for react)

Single-Step Forecast Accuracy (h = 1)

Metrics computed on test set:

ARMA → MAE=0.3701, RMSE=0.3788

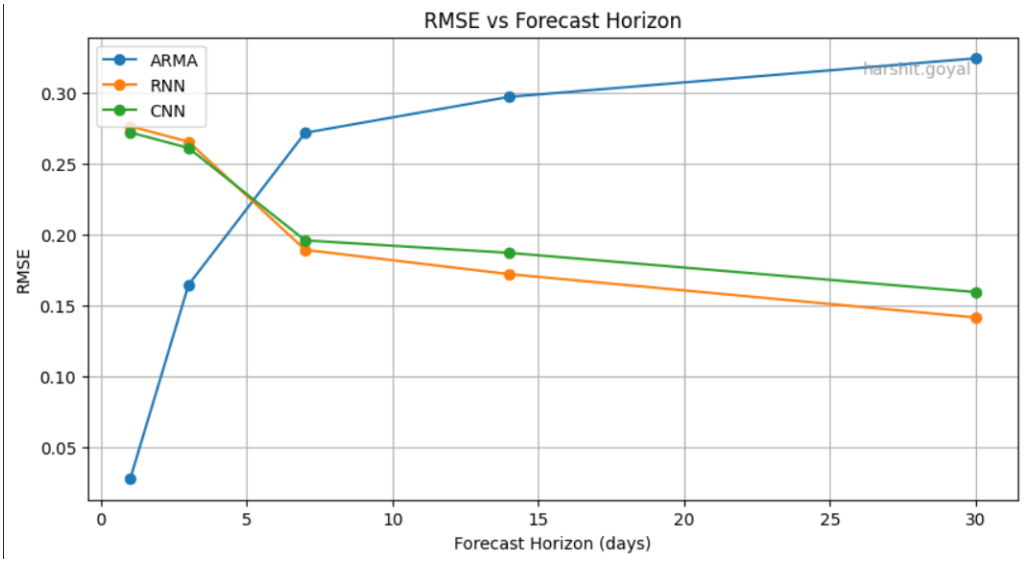
RNN → MAE=0.0433, RMSE=0.0811

CNN → MAE=0.0650, RMSE=0.1041

For immediate next-day forecasting, the RNN significantly outperforms both CNN and ARMA, demonstrating its advantage in modeling sequential dependencies and nonlinear temporal patterns in the star history.

Multi-Step Forecasting Analysis

The multi-step forecasting results show clear differences in model behavior. ARMA performs well only for immediate next-day predictions but rapidly loses accuracy as the forecast horizon increases. CNN remains stable across all horizons, showing moderate and consistent error. The RNN generalizes the best: after a small rise in RMSE at short horizons, it stabilizes and achieves the lowest errors for 15–30-day forecasts. Overall, RNN is the strongest long-horizon predictor, CNN is moderately robust, and ARMA is only suitable for short-term forecasts.



Summary of Generalization Behavior

- **Best long-horizon generalization: RNN**
- **Moderately stable generalization: CNN**
- **Poor long-horizon generalization: ARMA**

The results show that deep learning models especially the RNN generalize substantially better than classical ARMA when forecasting over extended horizons. CNN offers stable, balanced performance, while ARMA's accuracy deteriorates quickly as the forecast horizon grows.

4. Variational Autoencoder (VAE)

Dataset Preparation

Fashion-MNIST was loaded using PyTorch's torchvision API. Images were converted to tensors and normalized to $[-1,1]$using mean = 0.5 and std = 0.5. A batch size of 128 was used for efficient GPU training.

Model Architecture

A Variational Autoencoder (VAE) with latent dimension $z \in \mathbb{R}^2$was implemented.

Encoder: MLP → outputs μ and $\log\sigma^2$

Reparameterization: $z = \mu + \sigma \cdot \epsilon, \epsilon \sim \mathcal{N}(0, I)$

Decoder: MLP → reconstructs $\hat{x}$using sigmoid activation

We experimented with latent dimensions $z \in \mathbb{R}^2, \mathbb{R}^{10}, \mathbb{R}^{20}$and with shallower vs. deeper MLPs in the encoder/decoder. For very small latent dimension ($z = 2$), the latent space is highly smooth and well-organized (clear clusters, easy interpolation), but reconstruction fidelity is limited due to the strong bottleneck. Increasing the latent dimension ($z = 10,20$) improves reconstruction quality (sharper and more detailed images) at the cost of a less "compact" and less visually smooth latent space. Similarly, increasing network depth/width improves reconstruction fidelity by giving the model more capacity, but if the capacity is too large relative to the bottleneck, the model can partially ignore the KL regularization, leading to less-structured latent representations even though pixel-wise reconstructions look better.

the role of each term in shaping the learned latent representation

- The **reconstruction term** forces z to encode enough information to rebuild x, improving fidelity but potentially making the latent space irregular.
- The **KL term** regularizes q(z|x) toward the Gaussian prior, enforcing smoothness, continuity, and good generative sampling.
- Together, they balance **image quality** (reconstruction) and **latent space structure** (KL).

Effect of Frozen Latent Parameters

We compared samples generated by **freezing μ = 0 and varying σ ∈ {0.1, 0.5, 1.0}** to those obtained from **standard stochastic sampling** $z = \mu(x) + \epsilon \cdot \sigma(x)$from the encoder.

- For **fixed μ, σ = 0.1**, the samples are relatively **sharp but very similar to each other**. The latent noise is very small, so diversity is low and the model stays close to a small region around the origin in latent space.
- For **fixed μ, σ = 0.5**, there is a **better trade-off**: generated images show **moderate diversity** while still being reasonably **sharp and recognizable**. This setting roughly matches the scale seen during training.
- For **fixed μ, σ = 1.0**, the samples become much more **diverse**, but many images look **noisier and blurrier**. Large σ pushes z into regions of latent space that are less well supported by the training data, reducing visual quality.

  In contrast, **standard stochastic sampling** $z = \mu(x) + \epsilon \cdot \sigma(x)$from the encoder produces the **most realistic and class-consistent samples**, because each latent vector is centered around a data-dependent μ(x) with an appropriate σ(x). This preserves **image sharpness** and **semantic structure**, while still allowing some variability.
  Overall, freezing μ and σ reveals the trade-off: **small σ → sharp but low diversity**, **large σ → diverse but noisy**, whereas encoder-based sampling yields a **smooth, well-structured latent space** with good **reconstruction quality and generative realism**.